Data Cleaning & Preparation Report

Healthcare SQL Analysis | PostgreSQL | Scripts 01–04

Overview

Dataset	Healthcare Dataset (Kaggle — prasad22)
Tool	PostgreSQL via TablePlus
Raw rows	55,500
Clean rows	50,000 (5,500 duplicates removed)
Final schema	3 normalised tables: patients | admissions | insurance
Scripts covered	01_staging_table.sql · 02_data_cleaning.sql · 03_schema_creation.sql · 04_data_loading.sql

Stage 1 — Load Raw Data (01_staging_table.sql)

The raw CSV is loaded into a single flat staging table that mirrors the source file exactly. This staging approach preserves the original data and allows all cleaning steps to be performed and documented transparently before any normalisation takes place.

```
CREATE TABLE staging_healthcare (
    name VARCHAR(100), age INT,
    gender VARCHAR(10), blood_type VARCHAR(5),
    medical_condition VARCHAR(100),
    date_of_admission DATE, doctor VARCHAR(100),
    hospital VARCHAR(150), insurance_provider VARCHAR(100),
    billing_amount NUMERIC(12,2), room_number INT,
    admission_type VARCHAR(50), discharge_date DATE,
    medication VARCHAR(100), test_results VARCHAR(50)
);
```

55,500 rows loaded successfully ✅

Stage 2 — Data Profiling (02_data_cleaning.sql — Section 1)

Seven profiling checks were run across all columns before any cleaning took place.

1.1 Row Count

```
SELECT COUNT(*) AS total_rows FROM staging_healthcare;

-- Result: 55,500
```

1.2 NULL Check — All 15 Columns

```
SELECT
    COUNT(*) - COUNT(name) AS name_nulls,
    COUNT(*) - COUNT(age) AS age_nulls,
    COUNT(*) - COUNT(billing_amount) AS billing_nulls
    -- (all 15 columns checked)
FROM staging_healthcare;
```

Result: 0 NULLs across all columns ✅

1.3 Duplicate Check

```
SELECT name, date_of_admission, hospital, COUNT(*) AS duplicate_count
FROM staging_healthcare
GROUP BY name, date_of_admission, hospital
HAVING COUNT(*) > 1 LIMIT 20;
```

Result: Duplicates found — investigation required ⚠

1.4 Age Range Check

```
SELECT MIN(age), MAX(age), AVG(age) FROM staging_healthcare;
```

Result: Min 13, Max 89, Avg 51.54 — acceptable range ✅

1.5 Billing Range Check

```
SELECT MIN(billing_amount), MAX(billing_amount), AVG(billing_amount)
FROM staging_healthcare;
```

Result: Min -2,008.49 — negative values found ⚠

1.6 Categorical Value Check

```
SELECT DISTINCT gender FROM staging_healthcare;
SELECT DISTINCT blood_type FROM staging_healthcare;
SELECT DISTINCT admission_type FROM staging_healthcare;
SELECT DISTINCT test_results FROM staging_healthcare;
SELECT DISTINCT medical_condition FROM staging_healthcare;
```

Column	Distinct Values	Status
gender	Male, Female	✅
blood_type	A+ A- B+ B- O+ O- AB+ AB-	✅
admission_type	Elective, Urgent, Emergency	✅
test_results	Normal, Abnormal, Inconclusive	✅
medical_condition	Diabetes, Arthritis, Hypertension, Asthma, Obesity, Cancer	✅

1.7 Discharge Date Validation

```
SELECT COUNT(*) AS bad_dates FROM staging_healthcare
WHERE discharge_date < date_of_admission;
```

Result: 0 invalid date combinations ✅

Stage 3 — Data Cleaning (02_data_cleaning.sql — Sections 2–6)

Section 2 — Fix Inconsistent Text Casing

Columns affected: name, doctor

Rows affected: All rows

Visual inspection of the duplicate check results revealed random mixed capitalisation — for example "AARon smITh" and "ABIGAiL wateRS". Because PostgreSQL is case-sensitive, differently-cased versions of the same name were treated as different people, masking the true scale of the duplicate problem. INITCAP() was applied to name and doctor before any further cleaning.

```
UPDATE staging_healthcare SET name   = INITCAP(name);

UPDATE staging_healthcare SET doctor = INITCAP(doctor);



-- Verify

SELECT name, doctor FROM staging_healthcare LIMIT 10;
```

Section 3 — Remove Duplicate Rows

Rows removed: 5,500

After standardising casing, the duplicate check confirmed 5,500 true duplicate rows — records entirely identical across all columns. A row_id SERIAL column was added to allow safe deduplication, keeping the earliest occurrence of each record.

```
ALTER TABLE staging_healthcare ADD COLUMN row_id SERIAL;


DELETE FROM staging_healthcare

WHERE row_id NOT IN (

    SELECT MIN(row_id)

    FROM staging_healthcare

    GROUP BY name, date_of_admission, hospital

);


-- Verify: should return 0 rows

SELECT name, date_of_admission, hospital, COUNT(*)

FROM staging_healthcare

GROUP BY name, date_of_admission, hospital

HAVING COUNT(*) > 1;


-- Verify row count — Expected: 50,000

SELECT COUNT(*) AS total_rows FROM staging_healthcare;
```

Row count after deduplication: 50,000 ✅

Section 4 — Fix Negative Billing Amounts

Min value found: -$2,008.49

Negative billing amounts are not valid in a healthcare billing context. These were identified as data entry errors and corrected using ABS().

```
-- Inspect first

SELECT * FROM staging_healthcare WHERE billing_amount < 0;


-- Fix

UPDATE staging_healthcare

SET billing_amount = ABS(billing_amount)

WHERE billing_amount < 0;


-- Verify — Expected: 9.24

SELECT MIN(billing_amount) FROM staging_healthcare;
```

All billing values confirmed positive — Min: $9.24 ✅

Section 5 — Billing Outlier Check

```
ALTER TABLE staging_healthcare ADD COLUMN billing_outlier BOOLEAN DEFAULT FALSE;


UPDATE staging_healthcare SET billing_outlier = TRUE

WHERE billing_amount > (

    SELECT AVG(billing_amount) + 2 * STDDEV(billing_amount)

    FROM staging_healthcare

);


-- Result: 0 outliers flagged

SELECT COUNT(*) AS outlier_count FROM staging_healthcare

WHERE billing_outlier = TRUE;
```

0 billing outliers flagged beyond 2 standard deviations
✅

High billing amounts up to $52,764 are plausible for conditions such as cancer treatment and were retained as valid data.

Section 6 — Trim Whitespace from All Text Columns

TRIM() applied to all 10 text columns as a preventative measure to avoid silent JOIN failures during the normalisation step.

```
UPDATE staging_healthcare SET name               = TRIM(name);

UPDATE staging_healthcare SET gender             = TRIM(gender);

UPDATE staging_healthcare SET blood_type         = TRIM(blood_type);

UPDATE staging_healthcare SET medical_condition  = TRIM(medical_condition);

UPDATE staging_healthcare SET hospital           = TRIM(hospital);

UPDATE staging_healthcare SET insurance_provider = TRIM(insurance_provider);

UPDATE staging_healthcare SET admission_type     = TRIM(admission_type);

UPDATE staging_healthcare SET medication         = TRIM(medication);

UPDATE staging_healthcare SET test_results       = TRIM(test_results);

UPDATE staging_healthcare SET doctor             = TRIM(doctor);


-- Verify sample

SELECT name, gender, medical_condition FROM staging_healthcare LIMIT 5;
```

Section 7 — Final Verification

```
-- 7.1 Row count — Expected: 50,000
SELECT COUNT(*) AS final_row_count FROM staging_healthcare;


-- 7.2 Billing range — Expected Min: 9.24
SELECT MIN(billing_amount) AS min_bill, MAX(billing_amount) AS max_bill
FROM staging_healthcare;


-- 7.3 No duplicates — Expected: 0 rows
SELECT name, date_of_admission, hospital, COUNT(*)
FROM staging_healthcare
GROUP BY name, date_of_admission, hospital
HAVING COUNT(*) > 1;
```

All final verification checks passed ✅

Stage 4 — Schema Design & Normalisation (03_schema_creation.sql)

Hospital Name Quality — A Documented Design Decision

During normalisation, a hospitals table was initially planned as a 4th entity. However profiling revealed 39,876 ostensibly distinct hospital names across 50,000 rows — caused by fragmented and shuffled naming conventions in the source data:

Example 1	Scott Edwards, Rose And
Example 2	And Rose, Scott Edwards
Example 3	Roberts And Owens Scott,

These are the same hospitals with word order and punctuation randomised during data collection. Forcing normalisation on unreliable data would produce thousands of false hospital records. Hospital name was retained as a direct attribute on the admissions table. Normalisation should only be applied where source data quality reliably supports it.

Final 3-Table Schema

```
CREATE TABLE patients (
    patient_id SERIAL PRIMARY KEY, name VARCHAR(100),
    age INT, gender VARCHAR(10), blood_type VARCHAR(5)
);


CREATE TABLE admissions (
    admission_id SERIAL PRIMARY KEY,
    patient_id INT REFERENCES patients(patient_id),
    hospital_name VARCHAR(200), medical_condition VARCHAR(100),
    date_of_admission DATE, discharge_date DATE,
    admission_type VARCHAR(50), room_number INT,
    doctor VARCHAR(100), medication VARCHAR(100), test_results VARCHAR(50)
);


CREATE TABLE insurance (
    insurance_id SERIAL PRIMARY KEY,
    admission_id INT REFERENCES admissions(admission_id),
    provider_name VARCHAR(100), billing_amount NUMERIC(12,2)
);
```

Stage 5 — Data Loading (04_data_loading.sql)

The 3 tables were populated from the clean staging data. Patients were deduplicated by name, keeping the most recent age where a patient appeared in multiple years. The insurance JOIN required tightening across 4 columns to resolve patients with multiple admissions on the same date.

```
-- Patients: one row per unique name

INSERT INTO patients (name, age, gender, blood_type)

SELECT DISTINCT ON (name) name, age, gender, blood_type

FROM staging_healthcare ORDER BY name, age DESC;

-- Result: 40,235 unique patients


-- Insurance: 4-column JOIN to uniquely match each row

INSERT INTO insurance (admission_id, provider_name, billing_amount)

SELECT a.admission_id, s.insurance_provider, s.billing_amount

FROM staging_healthcare s

JOIN patients   p ON p.name              = s.name

JOIN admissions a ON a.patient_id        = p.patient_id

                 AND a.date_of_admission = s.date_of_admission

                 AND a.medical_condition = s.medical_condition

                 AND a.room_number       = s.room_number;

-- Result: 50,000
```

Table	Final Row Count	Notes
patients	40,235	40,235 unique patients — ~1.25 admissions per patient on average
admissions	50,000	Matches clean staging row count ✅
insurance	50,000	One billing record per admission ✅

Summary of All Issues Found & Resolved

#	Issue	Rows Affected	Action	Result
1	Inconsistent text casing — name & doctor	All rows	INITCAP()	✅
2	Duplicate rows	5,500 removed	MIN(row_id) deduplication	✅
3	Negative billing amounts	Several rows	ABS() conversion	✅
4	Whitespace in text columns	Preventative	TRIM() all text columns	✅
5	Unreliable hospital names	39,876 variants	Retained as attribute	✅
6	NULL values	None found	No action required	✅
7	Age outliers	None — range 13–89	Documented	✅
8	Billing outliers	None found	Documented	✅
9	Invalid discharge dates	None found	No action required	✅
10	Categorical inconsistencies	None found	No action required	✅

Clean dataset of 50,000 admissions across 40,235 unique patients — ready for analysis.